

1 Cơ học của gradient (Mechanics of the Gradient)

PHỤ THUỘC A/01-toan-toi-thieu (quy tắc chuỗi, Jacobian, chuẩn ma trận); hữu ích nhưng không bắt buộc: B/11-gioi-han-tinh-toan nếu muốn hiểu vì sao chỉ phí bộ nhớ ở cuối mục này lại là ràng buộc thật chứ không phải chỉ tiết phụ.

TÓM TẮT

Mục này giải thích vì sao thứ tự áp dụng quy tắc chuỗi quyết định chi phí huấn luyện, và vì sao **lan truyền ngược** (*backpropagation*) rẻ hơn vì phân thuận khoảng tám bậc độ lớn trên một mạng thật. Đọc xong bạn tự viết được lượt ngược cho một tầng bất kỳ. Bạn cũng tự bắt được lỗi của nó bằng kiểm tra gradient số, và ước lượng được bộ nhớ mà kiến trúc sẽ ngốn. Nếu chỉ nhớ một điều: lan truyền ngược không phải công thức riêng của mạng nơ-ron, nó là thuật toán vi phân cho mọi đồ thị khả vi, nên mọi kiến trúc lạ chỉ còn là một đồ thị mới.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, nhận đầu vào và trả về dự đoán; “huấn luyện” nghĩa là đi tìm bộ tham số làm dự đoán sát nhãn nhất.
- **tham số (parameter, weight)** — các con số bên trong mô hình mà quá trình huấn luyện được phép thay đổi; một mạng thị giác cỡ vừa có hàng chục triệu con số như vậy.
- **bias** — **phân biệt hai nghĩa** — trong công thức $Wx + b$, b là *tham số cộng* của một tầng. Từ “bias” trong thống kê lại nghĩa là *độ lệch hệ thống* của một bộ ước lượng. Mục này chỉ dùng nghĩa thứ nhất.
- **hàm mất mát (loss function)** — hàm biến dự đoán và nhãn thành một số duy nhất đo mức sai; mọi thứ trong huấn luyện đều nhằm giảm số đó.
- **gradient** — vectơ đạo hàm riêng của hàm mất mát theo từng tham số; nó chỉ hướng làm mất mát TĂNG nhanh nhất, nên ta đi ngược lại.
- **Jacobian** — ma trận đạo hàm riêng của một hàm nhiều đầu vào, nhiều đầu ra; nó là dạng tổng quát của gradient và là đối tượng chính mà lượt ngược thao tác.
- **tensor** — mảng số nhiều chiều — một ảnh RGB là tensor ba chiều, một lô ảnh là tensor bốn chiều; đây là đơn vị dữ liệu mà mọi phép toán trong mạng làm việc với.
- **batch (lô)** — nhóm mẫu được đưa qua mạng cùng lúc để tận dụng phần cứng song song; kích thước lô ảnh hưởng tới cả tốc độ lẫn chất lượng, và ta sẽ thấy vì sao ở các mục sau.
- **kích hoạt (activation)** — giá trị đầu ra của một tầng ở giữa mạng, tức dữ liệu đang trên đường đi; đừng nhầm với “hàm kích hoạt” là phép phi tuyến sinh ra nó.

A. Đặt vấn đề

Một mạng hiện đại là hàm hợp của vài nghìn phép toán sơ cấp. Nó phụ thuộc vào 10^7 đến 10^9 tham số vô hướng. Mỗi bước huấn luyện lại cần đạo hàm của *một* số duy nhất — giá trị **hàm mất mát** (*loss function*) — theo *toàn bộ* chỗ tham số đó. Câu hỏi không phải “quy tắc chuỗi có áp dụng được không”, vì nó luôn áp dụng được. Câu hỏi là: áp dụng quy tắc chuỗi *theo thứ tự nào* để chi phí là một lượt đi thay vì một trăm triệu lượt. Toàn bộ nội dung của **lan truyền ngược** (*backpropagation*) nằm trong câu hỏi thứ tự đó, chứ không nằm trong bản thân đạo hàm.

Bạn đã gặp mọi nguyên liệu của bài toán này ở phía tối ưu hoá phi tuyến trong SLAM: dựng Jacobian, nhân dồn các Jacobian cục bộ, giải hệ chuẩn tắc. Nhưng hình dạng bài toán ở đó ngược hẳn. Trong bundle adjustment, số dư (*residual*) rất nhiều còn tham số tương đối ít, nên việc dựng tường minh ma trận J rồi làm việc với $J^T J$ là hợp lý. Trong huấn luyện mạng, đầu ra chỉ có *một* chiều còn đầu vào có hàng trăm triệu chiều. **Forward-mode autodiff** lan truyền một đạo hàm theo hướng: mỗi lượt cho bạn đạo hàm theo đúng một hướng đầu vào, nên cần 10^8 lượt. **Reverse-**

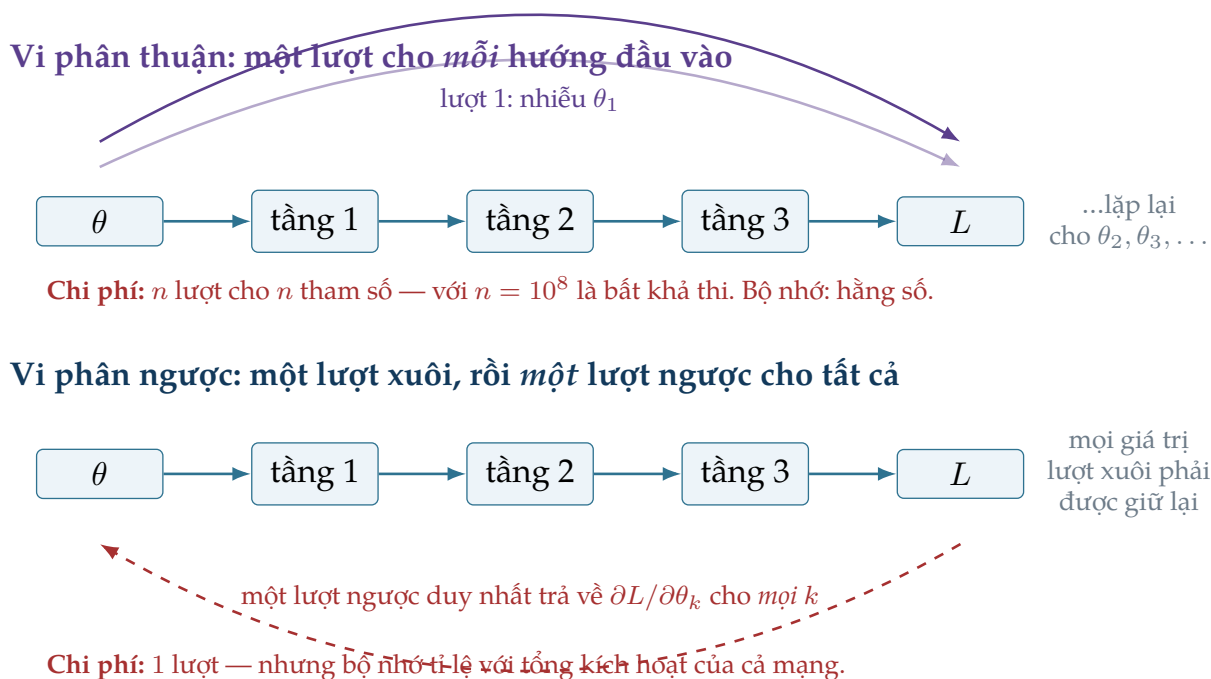
mode autodiff lan truyền độ nhảy ngược từ đầu ra: mỗi lượt cho bạn đạo hàm theo *tất cả* đầu vào, nên cần đúng một lượt cho mỗi đầu ra vô hướng. Tỉ số 10^8 giữa hai cách là lý do tồn tại của lan truyền ngược [1]. Cùng một quy tắc chuỗi, hai thứ tự áp dụng, chi phí lệch nhau tám bậc độ lớn.

Mục này cho bạn ba việc cụ thể. Bạn tự viết được lượt ngược cho một tầng chưa có trong thư viện. Bạn tự bắt được lỗi của nó bằng kiểm tra số. Và bạn ước lượng được bộ nhớ mà một kiến trúc sẽ ngốn. Trước đó cả ba đều phải tin vào framework.

Bảng 1. Hai chế độ vi phân tự động. Chúng dùng đúng cùng một quy tắc chuỗi và chỉ khác nhau ở thứ tự nhân các Jacobian, nhưng thứ tự đó quyết định toàn bộ chi phí.

Chế độ	Cơ chế	Chi phí cho n đầu vào, m đầu ra	Hỏng / không dùng khi
Forward-mode	Lan truyền đạo hàm theo một hướng đầu vào, cùng chiều lượt xuôi	n lượt; bộ nhớ hằng số	n lớn — huấn luyện mạng có 10^8 tham số là bất khả thi
Reverse-mode	Lan truyền độ nhảy của một đầu ra, ngược chiều lượt xuôi	m lượt; bộ nhớ tỉ lệ tổng kích hoạt	m lớn (cần cả Jacobian đầy đủ), hoặc bộ nhớ không đủ giữ kích hoạt

Bảng 1 nói gọn lại: huấn luyện là bài toán “một đầu ra, rất nhiều đầu vào”, và đó chính xác là hình dạng mà reverse-mode được sinh ra để phục vụ.



Hình 1. Cùng một quy tắc chuỗi, hai thứ tự nhân, hai chi phí lệch nhau tám bậc độ lớn. Vi phân thuận đi cùng chiều lượt xuôi nên mỗi lượt chỉ trả lời được cho một hướng đầu vào; vi phân ngược đi ngược lại nên một lượt phủ mọi tham số. Điểm đáng nhớ nằm ở dòng cuối mỗi nửa hình: cái ta tiết kiệm được ở thời gian phải trả lại bằng bộ nhớ.

Hình 1 cho thấy điều mà bảng không nói được: hai chế độ dùng *cùng* một đồ thị và cùng các Jacobian cục bộ, chỉ khác chiều duyệt — và chính chiều duyệt sinh ra cả lợi thế lẫn cái giá.

Nhận thức then chốt cần mang ra khỏi mục này là: lan truyền ngược không phải công thức riêng của mạng nơ-ron. Nó là thuật toán chung cho *mọi* đồ thị có hướng không chu trình mà mỗi nút khả vi. Khi bạn đã tin điều đó, một kiến trúc lạ không còn là một thứ phải học thuộc; nó chỉ là một đồ thị mới, và bạn đã biết cách vi phân mọi đồ thị.

TRỰC GIÁC

Hãy nghĩ về forward pass như một dây chuyền sản xuất: nguyên liệu đi qua ba mươi trạm, mỗi trạm biến đổi một chút, cuối dây chuyền ra một sản phẩm có một con số sai lệch so với chuẩn. Forward-mode giống việc rung nhẹ *một* nút vặn rồi chạy lại cả dây chuyền để xem sai lệch đổi bao nhiêu. Cách đó đúng, nhưng phải lặp lại đúng bằng số nút vặn. Reverse-mode làm ngược: bắt đầu từ sản phẩm lỗi, hỏi trạm cuối “một đơn vị sai ở đây tương ứng với bao nhiêu sai ở đầu vào của anh”, rồi chuyển đúng câu hỏi đó ngược lên trạm trước. Một lượt đi ngược duy nhất trả lời cho mọi nút vặn trên toàn dây chuyền, vì mỗi trạm chỉ cần biết *quy tắc quy đổi cục bộ* của riêng nó.

B. Đồ thị tính toán và tích vector-Jacobian (computational graph, VJP)

Hình thức hoá của ẩn dụ trên rất gọn. Biểu diễn phép tính thành một đồ thị: nút là giá trị trung gian, cạnh là phụ thuộc. Lướt xuôi duyệt đồ thị theo thứ tự tô-pô và tính giá trị; lướt ngược duyệt theo thứ tự tô-pô đảo và tính *adjoint* $\bar{v} = \partial L / \partial v$ cho từng nút. Mỗi nút chỉ cần biết một thứ: cách biến adjoint ở đầu ra của nó thành adjoint ở đầu vào của nó. Nếu nút thực hiện $y = g(x)$ thì quy tắc đó là

$$\bar{x} = \left(\frac{\partial y}{\partial x} \right)^\top \bar{y},$$

tức là một **tích vector-Jacobian** (*vector-Jacobian product, VJP*). Điểm quan trọng về mặt kỹ thuật là ta *không bao giờ* dựng ma trận Jacobian ở giữa. Xét một tầng affine $y = Wx + b$ với $W \in \mathbb{R}^{4096 \times 4096}$. Jacobian theo W là ma trận $4096 \times 16,8$ triệu, khoảng $6,9 \cdot 10^{10}$ phần tử. Không thể lưu nổi. Nhưng VJP của nó chỉ là ba biểu thức tầm thường:

$$\bar{x} = W^\top \bar{y}, \quad \bar{W} = \bar{y} x^\top, \quad \bar{b} = \bar{y},$$

trong đó $\bar{W}$ là một tích ngoài với đúng 16,8 triệu phần tử — bằng kích thước của chính W . Tỉ số 4096 giữa “dựng Jacobian” và “áp dụng Jacobian” là lý do mọi framework *autodiff* đều cài đặt VJP chứ không cài đặt đạo hàm theo nghĩa ma trận. Với một batch X xếp theo hàng, quy tắc trên trở thành $\bar{W} = \bar{Y}^\top X$, và bạn nên coi đây là công thức phải viết lại được từ đầu bất cứ lúc nào.

↔ **trường hợp riêng của A/01** — lan truyền ngược chỉ là quy tắc chuỗi của giải tích nhiều biến, áp dụng theo thứ tự làm chi phí rẻ nhất

↔ **tiền đề C/12/02** — chính tích các Jacobian cục bộ ở đây là đại lượng mà ba hướng chống gradient tiêu biến nhắm vào

C. Ba mẫu gradient lặp lại (the three gradient patterns)

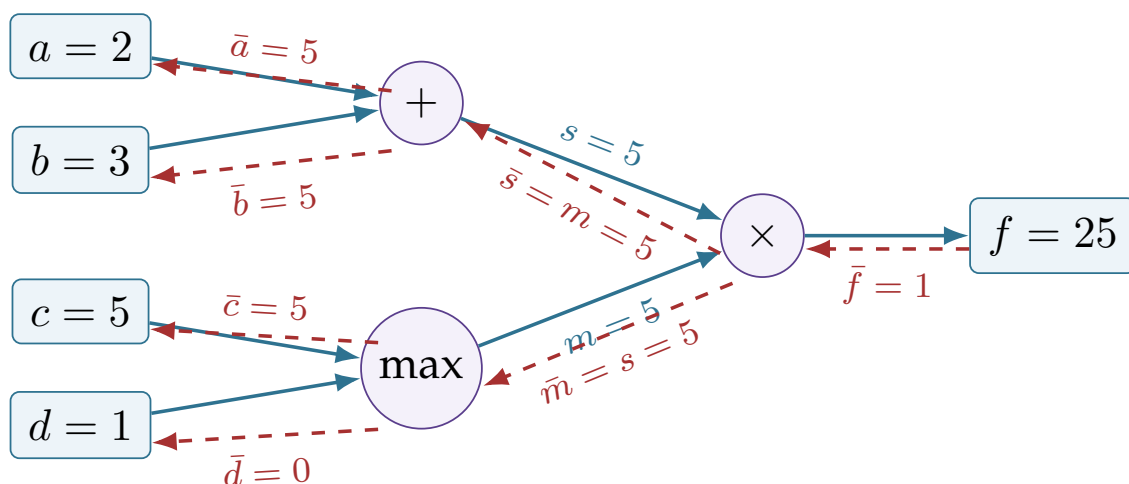
Nghe qua thì việc phải derive backward cho từng loại tầng là công việc vô tận. Thực tế thì không, vì gần như mọi đồ thị đều được ghép từ ba mẫu, và ba mẫu đó có tên gọi trực quan.

- **Cộng là bộ phân phối** (*add = distributor*) — nếu $y = u + v$ thì $\bar{u} = \bar{v} = \bar{y}$. Nói gọn: adjoint đi qua nút cộng mà không đổi giá trị, chỉ nhân bản ra mọi nhánh.
- **Nhân là bộ hoán đổi** (*multiply = swapper*) — nếu $y = uv$ thì $\bar{u} = \bar{v} \bar{y}$ và $\bar{v} = \bar{u} \bar{y}$. Mỗi nhánh nhận adjoint nhân với giá trị của *nhánh kia*; đây chính là lý do lướt ngược buộc phải nhớ các giá trị của lướt xuôi, và do đó là gốc rễ của chi phí bộ nhớ ở mục G.

- **Max là bộ định tuyến** ($\text{max} = \text{router}$) — nếu $y = \max(u, v)$ thì toàn bộ adjoint đi về nhánh thắng, nhánh thua nhận 0. Nói gọn: max không trộn, nó chọn đường.
- **Rẽ nhánh là bộ cộng dồn** ($\text{fan-out} = \text{accumulator}$) — mẫu thứ tư, ít được nêu tên nhưng gây nhiều lỗi nhất. Nếu một giá trị được dùng ở hai chỗ trong đồ thị, adjoint của nó là tổng các adjoint quay về từ hai chỗ đó.

Mẫu thứ tư đáng được nói thêm một câu vì hệ quả thực hành của nó rất rộng: *lượt ngược của broadcasting là một phép cộng theo đúng trục đã được broadcast*. Cộng một vector bias shape $(C,)$ vào một *tensor* shape (N, C, H, W) chính là rẽ nhánh giá trị bias ra $N \cdot H \cdot W$ chỗ, nên $\bar{b} = \sum_{n,h,w} \bar{y}_{n,c,h,w}$. Một mình quy tắc này giải thích phần lớn lỗi shape mà người mới gặp khi tự viết lượt ngược.

Hình 2 chạy cả ba mẫu trên một ví dụ đủ nhỏ để tính tay. Với $f = (a + b) \cdot \max(c, d)$ và $a = 2, b = 3, c = 5, d = 1$, lượt xuôi cho $s = 5, m = 5, f = 25$. Lượt ngược đặt $\bar{f} = 1$; nút nhân hoán đổi nên $\bar{s} = m = 5$ và $\bar{m} = s = 5$; nút cộng phân phối nên $\bar{a} = \bar{b} = 5$; nút max định tuyến nên $\bar{c} = 5$ và $\bar{d} = 0$. Kiểm tra bằng trực giác: tăng d một chút không đổi gì cả vì d đang thua, đúng như $\bar{d} = 0$; nhưng nếu d vượt c thì đạo hàm nhảy bậc, và đó chính là chỗ kiểm tra gradient bằng số sẽ báo động giả ở phần sau.



Hình 2. Ba mẫu gradient trên một đồ thị bốn nút. Mũi tên liền là lượt xuôi, mũi tên đứt là lượt ngược. Nút cộng phân phối adjoint, nút nhân hoán đổi giá trị hai nhánh, nút max định tuyến toàn bộ adjoint về nhánh thắng và cắt nhánh thua về 0.

D. Bốn backward đáng tự viết một lần trong đời

Việc tự derive gradient cho vài tầng cụ thể là bài tập kiểu CS231n, và nó nằm ở tầng bán rã [T2]–[T3]: không phải kiến thức vĩnh cửu, nhưng cũng không phải chi tiết framework sẽ mất giá sau một năm. Giá trị của nó là sau khi làm xong, bạn đọc bất kỳ tầng mới nào cũng tự hỏi “VJP của cái này là gì” thay vì “thư viện có hỗ trợ không”.

Tầng affine đã nêu ở trên. **ReLU** là mẫu max trá hình: $\bar{x} = \bar{y} \cdot \mathbb{I}[x > 0]$, và lượt ngược chỉ cần một mặt nạ bit. **Softmax ghép cross-entropy** là ví dụ đẹp nhất về việc ghép hai phép toán làm backward sụp đổ: tính riêng thì Jacobian của softmax là $\text{diag}(p) - pp^\top$, một ma trận $K \times K$ phải nhân với adjoint; ghép lại thì tất cả rút gọn thành $\bar{z} = p - y$, vừa rẻ hơn vừa ổn định hơn về số học. **Max pooling** định tuyến adjoint về vị trí argmax nên phải lưu chỉ số argmax từ lượt xuôi — một chi phí bộ nhớ thật — trong khi average pooling chỉ phân phối $1/k^2$ và không cần nhớ gì.

BatchNorm là bài khó nhất và cũng bổ ích nhất, vì nó là tầng đầu tiên mà gradient của một mẫu phụ thuộc vào *mọi mẫu khác trong batch*. Chuỗi phép toán là trung bình $\rightarrow$ phương sai $\rightarrow$ chuẩn hoá $\rightarrow$ scale-and-shift, và vì μ lẫn σ^2 đều là hàm của cả batch nên adjoint chạy ngược qua ba

đường song song rồi cộng lại. Kết quả gọn có dạng

$$\bar{x}_i = \frac{\gamma}{N\sigma} \left(N \hat{x}_i - \sum_j \hat{x}_j - \hat{x}_i \sum_j \hat{x}_j \hat{x}_j \right),$$

với $\hat{x}$ là giá trị đã chuẩn hoá. Hai tổng trong ngoặc chính là hai đường phụ thuộc qua μ và qua σ^2 . Công thức này giải thích luôn một câu hỏi hay bị hỏi: vì sao BatchNorm làm lượt ngược tốn thêm bộ nhớ. Vì để tính được nó, lượt xuôi phải giữ lại $\hat{x}$ và σ cho mỗi tầng BN, chứ không thể vứt đi sau khi dùng.

→ **ràng buộc bởi** C/12/04 — việc gradient một mẫu phụ thuộc cả batch là gốc của mọi rắc rối BatchNorm trong huấn luyện phân tán

E. Kiểm tra gradient bằng số (numerical gradient check)

Derive tay thì sai lúc nào không biết, và điều tệ là một gradient sai *vẫn làm loss giảm* — chỉ là giảm chậm hơn hoặc hội tụ về chỗ khác. Nên cần một phép kiểm tra độc lập. Công cụ là **sai phân hữu hạn** (*finite difference*) đối xứng

$$\frac{\partial f}{\partial x_k} \approx \frac{f(x + he_k) - f(x - he_k)}{2h},$$

chứ không phải sai phân một phía. Lý do là khai triển Taylor: sai phân một phía có sai số bậc $O(h)$, sai phân đối xứng triệt tiêu số hạng bậc hai nên sai số bậc $O(h^2)$. Với $h = 10^{-5}$, đó là chênh lệch giữa sai số cỡ 10^{-5} và cỡ 10^{-10} — đủ để phân biệt “code sai” với “xấp xỉ số”.

Tiêu chí so sánh phải là sai số tương đối $|a - n| / \max(|a|, |n|)$ chứ không phải sai số tuyệt đối, vì độ lớn gradient thay đổi hàng chục bậc giữa các tầng. Kinh nghiệm thực hành phổ biến: dưới 10^{-7} là tốt, quanh 10^{-4} là đáng nghi trừ khi hàm có điểm gãy, trên 10^{-2} gần như chắc chắn sai. Ba điều kiện phải thoả thì con số đó mới có nghĩa. Thứ nhất, phải tính bằng *double*: với float32, epsilon máy khoảng $1,2 \cdot 10^{-7}$, nên phép trừ hai giá trị gần nhau ở tử số mất gần hết chữ số có nghĩa và bạn đang đo nhiễu làm tròn chứ không đo đạo hàm. Thứ hai, phải tắt mọi nguồn ngẫu nhiên — dropout, augmentation, thứ tự batch — hoặc cố định seed, vì f phải là một hàm tắt định thì sai phân mới có nghĩa. Thứ ba, chỉ kiểm vài chục toạ độ ngẫu nhiên, không kiểm hết, vì mỗi toạ độ tốn hai lượt xuôi.

Điểm gãy là cái bẫy lớn nhất. Giả sử x_k rất gần 0 ở đầu vào một ReLU. Khi đó $x_k + h$ và $x_k - h$ nằm hai bên điểm gãy. Sai phân số trả về một giá trị trung gian, còn gradient giải tích trả về 0 hoặc 1. Kiểm tra “thất bại” dù code hoàn toàn đúng. Cách xử lý là dùng ít điểm dữ liệu, chọn toạ độ ở xa điểm gãy, và diễn giải một vài lần thất bại cô lập một cách thận trọng thay vì lao đi viết lại backward. Kỹ năng này không hề mất giá theo thời gian: mỗi lần bạn viết một autograd.Function riêng hay một kernel CUDA cho một phép toán mới, việc đầu tiên nên làm là chạy `torch.autograd.gradcheck` với đầu vào double.

F. Vectorization, và vì sao nó không chỉ là chuyên tốc độ

Bỏ vòng lặp Python khỏi loss và gradient là bài tập kinh điển. Lợi ích hiển nhiên là tốc độ, nhưng chi phí không nằm ở FLOPs. Nó nằm ở phần trên đầu của mỗi lời gọi: thông dịch Python cộng chi phí phóng một kernel, cỡ vài micro-giây. Một nghìn phép toán tí hon vì thế tiêu tốn vài mili-giây trước khi làm phép nhân nào. Đây đúng là hiện tượng bạn đã đo khi tối ưu SLAM: nghẽn ở số lời gọi chứ không ở số phép tính. Nhưng lợi ích thứ hai mới đáng giá lâu dài: viết dạng vector buộc bạn phát biểu rõ shape của mọi thứ, do đó phát biểu rõ *ai rẽ nhánh sang ai*, và khi đó backward gần như tự lộ ra từ ba mẫu ở trên. Đoạn dưới là toàn bộ forward và backward của softmax ghép cross-entropy cho một batch.

```
# Z: (N, K) logits, y: (N,) nhãn đúng
Z = Z - Z.max(axis=1, keepdims=True) # on đỉnh so: tránh exp tràn
P = np.exp(Z); P /= P.sum(axis=1, keepdims=True)
loss = -np.log(P[np.arange(N), y]).mean()

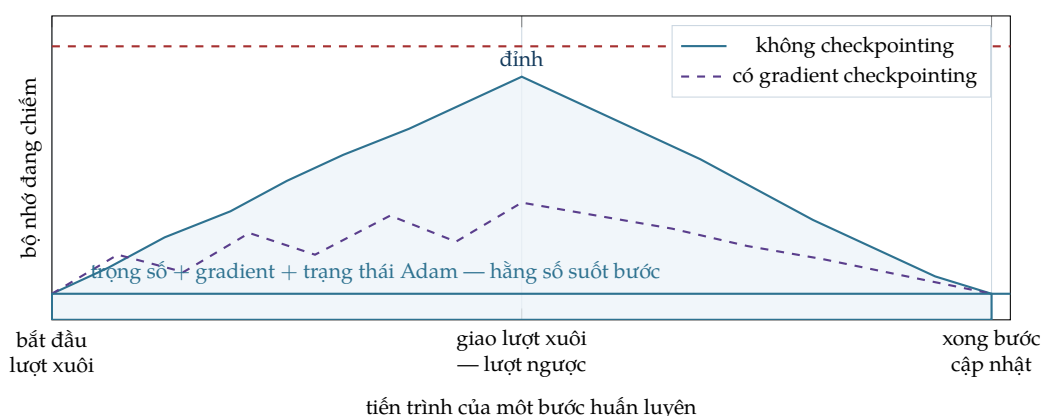
dZ = P.copy()
dZ[np.arange(N), y] -= 1.0 # dZ = p - y
dZ /= N # vì loss lấy trung bình theo batch
dW = X.T @ dZ # VJP của tang affine
db = dZ.sum(axis=0) # backward của broadcast = cộng theo trục
```

Hai dòng cuối minh họa chính xác hai quy tắc đã nêu: tích ngoài cho ma trận trọng số, và tổng theo trục batch cho bias.

G. Cái giá của reverse mode: bộ nhớ kích hoạt

Reverse-mode rẻ về thời gian nhưng không miễn phí, và cái giá của nó là bộ nhớ. Mẫu nhân đòi hỏi giá trị của nhánh kia. Nên lượt ngược cần phần lớn giá trị trung gian của lượt xuôi. Bộ nhớ vì thế tăng theo tổng kích thước **kích hoạt** (*activation*) của cả mạng, không theo số tham số. Một phép tính cụ thể cho thấy bậc độ lớn. Lấy đầu ra của một tầng conv ở batch 32, 64 kênh, độ phân giải 112×112 , lưu ở fp32. Nó chiếm $32 \cdot 64 \cdot 112 \cdot 112 \cdot 4 \approx 1,03 \cdot 10^8$ byte, tức khoảng 98 MiB cho một tầng. Một backbone vài chục tầng nhanh chóng đưa con số này lên vài GB. Nên trong phần lớn bài huấn luyện thị giác, thứ quyết định batch size tối đa là bộ nhớ chứ không phải tính toán.

↪ **ràng buộc bởi** B/11 — bộ nhớ kích hoạt là đại lượng chặn batch size, và chương giới hạn tính toán đo nó cùng với bảng thông trên mô hình roofline



Hình 3. Bộ nhớ trong một bước huấn luyện. Phần không đổi là trọng số, gradient và trạng thái optimizer; phần dâng lên rồi rút xuống là kích hoạt, và nó đạt đỉnh đúng ở chỗ lượt xuôi giao lượt ngược — vì đó là lúc mọi giá trị trung gian đã được sinh ra mà chưa có giá trị nào được giải phóng. Hai hệ quả đọc thẳng từ hình: batch size tối đa bị quyết định bởi một thời điểm chứ không phải bởi mức trung bình, và checkpointing hạ đỉnh bằng cách vớt rồi tính lại, đổi lấy phần diện tích tăng thêm ở lượt ngược. Các con số là minh họa định tính, không phải số đo.

Hình 3 giải thích một điều mà con số 98 MiB ở trên không nói: cái chặn bạn không phải tổng bộ nhớ trung bình mà là *đỉnh tức thời* tại điểm giao giữa hai lượt. Đó cũng là lý do một mô hình chạy được ở batch 16 có thể sập ở batch 17 chứ không xuống cấp dần.

Đây là một đánh đổi thời gian-bộ nhớ quen thuộc, và nó có lời giải quen thuộc. **Gradient checkpointing** [2] chỉ lưu một tập con các nút, rồi khi lượt ngược cần một giá trị đã bị vớt thì tính lại nó từ điểm lưu gần nhất; với cách chọn điểm lưu hợp lý, bộ nhớ giảm từ $O(L)$ xuống $O(\sqrt{L})$ với giá khoảng một lượt xuôi phụ, tức chừng 30 phần trăm thời gian. Ở đầu kia của phổ, forward-mode có bộ nhớ hằng số nhưng cần một lượt cho mỗi hướng đầu vào, nên nó không dùng để huấn

luyện mà dùng cho tích Jacobian-vector, chẳng hạn khi cần tích Hessian-vector trong các phương pháp bậc hai.

H. Giới hạn và trường hợp hỏng

Khung đồ thị có ba chỗ gãy đáng biết trước.

Thứ nhất là *điểm không khả vi*. ReLU tại 0 và max khi hai nhánh bằng nhau không có đạo hàm; framework chọn một quy ước, thường là 0 cho ReLU và “nhánh chỉ số nhỏ hơn thắng” cho max. Quy ước đó vô hại về mặt tối ưu hoá nhưng có hệ quả về tính tái lập: khi hai giá trị bằng nhau tới từng bit, việc ai thắng phụ thuộc thứ tự duyệt, và thứ tự duyệt có thể phụ thuộc lịch trình luồng. Đây đúng là loại nguồn bất định mà bạn đã truy trong hệ SLAM đa luồng, chỉ là xuất hiện ở một tầng khác.

Thứ hai là *đồ thị bị cắt mà không ai báo*. Một phép gán tại chỗ ghi đè giá trị mà lượt ngược cần sẽ cho gradient sai; PyTorch bắt được phần lớn trường hợp này bằng bộ đếm phiên bản tensor, nhưng một kernel tự viết thì không có ai bảo vệ và bạn nhận gradient sai một cách im lặng. Cùng loại đó là việc vô tình gọi `detach()` hoặc đặt một nhánh trong `no_grad()`: loss vẫn giảm, chỉ có một mô-đun không bao giờ học, và triệu chứng duy nhất là một đường cong hơi tệ hơn kỳ vọng.

Thứ ba là *ổn định số*. $\log(0)$, exp tràn, chia cho phương sai gần 0: ba nguồn NaN kinh điển. Chúng không phải lỗi của autodiff mà là lỗi của biểu thức bạn viết ra. Cách chữa luôn là viết lại biểu thức ở dạng ổn định: trừ max trước khi lấy exp, dùng `logsumexp`, cộng ϵ vào mẫu số. Kẹp gradient sau khi nó đã thành NaN thì quá muộn. Cùng họ với nó là điều khiển luồng phụ thuộc dữ liệu: một câu lệnh rẽ nhánh theo giá trị tensor là hợp lệ, nhưng gradient chỉ chảy qua nhánh đã chọn, nên nếu bạn tưởng mình đang huấn luyện cả hai nhánh thì bạn nhầm.

CẠM BÃY

Cái bẫy sinh ra nhiều gradient sai âm thầm nhất không phải công thức khó, mà là quên rằng *broadcast ở lượt xuôi là rẽ nhánh, nên lượt ngược phải cộng theo trục đã broadcast*. Nguy hiểm ở chỗ nếu bạn quên phép cộng đó, shape của kết quả nhiều khi vẫn hợp lệ nhờ chính broadcast, nên không có exception nào được ném ra; bạn chỉ nhận một gradient sai hệ số N và một quá trình huấn luyện chậm hơn đáng kể. Cách phòng duy nhất đáng tin là kiểm tra bằng số trên một ví dụ có $N = 3$ và $N = 1$ rồi so hai kết quả.

CẦN NHỚ

Lan truyền ngược = duyệt đồ thị theo thứ tự tô-pô đảo, mỗi nút áp một tích vector-Jacobian cục bộ, với ba mẫu cộng-nhân-max cộng thêm quy tắc “rẽ nhánh thì cộng dồn”. Nếu bạn gọi tên được Jacobian cục bộ của một khối, bạn vi phân được khối đó — kể cả khối bạn chưa từng thấy. [T1]

I. Kết nối

Mục này cho bạn công cụ tính gradient; mục tiếp theo, C/12/02-lam-cho-training-chay-duoc, hỏi câu kế: gradient tính đúng rồi thì làm sao để nó không chết hoặc nổ khi đi qua năm mươi tầng — và câu trả lời bắt đầu chính từ tích các Jacobian cục bộ vừa nêu ở đây. Chi phí bộ nhớ kích hoạt ở cuối mục là tiền đề trực tiếp cho B/11-gioi-han-tinh-toan (roofline, chế độ nghẽn bộ nhớ) và cho phần huấn luyện phân tán ở C/12/04-tong-quat-hoa-va-chan-doan. Ý “mọi kiến trúc chỉ là một đồ thị” được dùng lại nguyên vẹn ở C/13-kien-truc-backbone: khi đọc ResNet hay Transformer, thứ đáng hỏi không phải sơ đồ khối mà là đường đi của adjoint qua sơ đồ đó.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao reverse-mode rẻ hơn forward-mode đúng bằng tỉ số giữa số đầu vào và số đầu ra, và trong tình huống nào tỉ số đó đảo chiều khiến forward-mode thắng?
2. Vì sao không bao giờ dựng ma trận Jacobian của một tầng affine, dù về mặt toán học nó tồn tại?
3. Kiểm tra gradient bằng số báo sai số tương đối $3 \cdot 10^{-3}$ trên một mạng có ReLU — hai nguyên nhân khả dĩ là gì, và bạn phân biệt chúng bằng cách nào?
4. Vì sao gradient của BatchNorm với một mẫu lại phụ thuộc vào các mẫu khác trong batch, và điều đó kéo theo hệ quả gì cho huấn luyện phân tán?
5. Bạn viết một kernel CUDA cho một phép toán mới và lượt ngược cho gradient sai đúng hệ số bằng batch size. Chỗ nào trong code là vi phạm số một?
6. Gradient checkpointing đổi cái gì lấy cái gì, và khi nào phép đổi đó *không* đáng?

Nguồn

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., "Training Deep Nets with Sublinear Memory Cost," *arXiv:1604.06174*, 2016.